

ECE391: Computer Systems Engineering Machine Problem 3

Spring 2025

Checkpoint 1: Mon April 7 18:00 CDT

Checkpoint 2: Mon April 28 18:00 CDT

Checkpoint 3: Sun May 4 21:00 CDT

Illinux 391

Contents

1	Introduction	4
2	Using the Group Repository	4
3	The Pieces	5
3.1	Getting started	5
3.2	Work Plan	5
4	Testing	6
5	What to Hand in	6
5.1	Checkpoint 1: Filesystem and Drivers and Program Loading, (Oh My!)	6
5.1.1	Group Repository	6
5.1.2	VIRTIO Block Device	6
5.1.3	Read-Only Filesystem	7
5.1.4	Your Friendly Neighborhood Memio	8
5.1.5	Locked and	8
5.1.6	...(ELF) Loaded	9
5.1.7	Cache Me Outside	9
5.1.8	Existing Files	10
5.1.9	Running Your Code	11
5.1.10	Troubleshooting/Debugging	11
5.1.11	Checkpoint 1 Handin	11
5.2	Checkpoint 2: Virtual Memory and Process Abstraction	11

5.2.1	Given Code	11
5.2.2	Vicious Virtual Memory	12
5.2.3	Pesky Processes	13
5.2.4	Suspicious Syscalls	15
5.2.5	Fully Implemented Filesystem	16
5.2.6	Additional Modifications	17
5.2.7	Checkpoint 2 Handin	18
6	Grading	19
7	Appendix A: The File System	20
7.1	File System Overview	20
7.2	File Abstractions	22
7.3	Building the File System	23
7.4	The Blob	24
8	Appendix B: I/O Devices	25
8.1	Overview	25
8.2	I/O Operations	25
8.3	I/O Diagram	26
9	Appendix C: The Process Abstraction	27
9.1	History	27
9.2	Overview	27
9.3	Context Switching between User-Mode and Supervisor-Mode	27
10	Appendix D: RISC-V Paging	30
10.1	Illinix Mappings	30
10.2	Sv39	31
11	Appendix H: Troubleshooting	32
11.1	Debugging with gdb	32
11.1.1	Debugging the Kernel	32
11.1.2	Debugging User Programs	33

11.1.3 Useful gdb Commands	33
--------------------------------------	----

1 Introduction

Read the whole document before you begin, or you may miss points on some requirements.

In this machine problem, you collaborate to develop the core of an operating system roughly based on Unix Version 6, with modern concepts peppered in where appropriate. You'll implement interrupt logic, user threading (à la MP2), kernel and application paging, initialize some devices and a filesystem with the VIRTIO interface, and create a system call interface to support various system calls. The operating system will support running several tasks ("threads") spawned by a number of user programs; programs will interface with the kernel via system calls.

Don't worry, these aren't all "boring" programs—you'll get to run some cool games, too.

The goal for the assignment is to provide you with hands-on experience in developing the software used to interface between devices and applications, *i.e.*, operating systems. You should notice that the work here builds on concepts from the other machine problems. Many of the abstractions used here (*e.g.*, the VIRTIO interface) have been simplified to reduce the effort necessary to complete the project, but we hope that you will leave the class with the skills necessary to extend the implementation that you develop here along whatever direction you choose, by incrementally improving various aspects of your system.

2 Using the Group Repository

You should receive access to a shared repository with your group members. Your group has a two-digit group number; your group number should be released in a Piazza post, and your repository should be named something like `mp3_group_xx`, where `xx` is your number.

It is required to run the following commands on each computer that you clone the repository to, in order to make sure the line endings are set to LF (Unix style):

```
git config --global core.autocrlf input
git config --global core.eol lf
```

Some other tips:

As you work on MP3 with your teammates, you may find it useful to create additional branches to avoid conflicts while editing source files. Remember to `git pull` each time you sit down to work on the project and `git commit` and `git push` when you are done. Doing so will ensure that all members are working on the most current version of the sources. It is highly likely that you will benefit from proper usage of the `git stash` command, to correctly retain desired (local) changes when doing a pull.

When using Git, keep in mind that it is, in general, bad practice to commit broken sources. You should make sure that your sources compile correctly before committing them to the repository. Make sure not to commit compiled `*.o` files, besides what we have given you. You can modify your `.gitignore` in any way you want.

Finally, merge your changes into the `main` branch by each checkpoint deadline, as **this is the only branch we will use for grading.**

3 The Pieces

The basic OS is provided, but when you first get it, it won't properly do a lot of the things we expect an OS to do: perhaps most importantly [†], it can't play any cool games!

For simplicity, we will stick to text-mode graphics (for the most part), but your OS will, by the end, run the games from previous MPs as well as some new ones. We've included a few helpful pieces that will allow you to debug easier, such as `kprintf`. We **highly** encourage use of `gdb` to debug. Print statements can only take you so far. See **Appendix H** for details.

3.1 Getting started

In order to effectively work on this MP, you will need to develop knowledge of a lot of different and difficult concepts. The documentation on the course website and lectures will give you background information, but the best way to learn is to read and understand the code we have provided. This includes the `Makefiles`, `.ld` linker files, and `.c/.h` source files.

This MP is difficult, which is why we do not expect you to work alone. While you cannot share code or discuss details with other groups or anyone outside your group, you should work together with your team to get unstuck and build a common understanding.

3.2 Work Plan

“Work expands so as to fill the time available for its completion.” - Cyril Parkinson

This project is somewhat daunting, and will require efforts from all your team members. You should partition the work accordingly to allow independent progress by all team members.

Setting up a clean testing interface will also help substantially with partitioning the work, since group members can finish and test components before your groupmates finish the other parts (yet). The abstractions suggested should allow for some spots where a “working part” can be substituted with a functionally equivalent placeholder, so to speak—more on that later.

While splitting up the work allows for you to make more progress, it is still crucial that you spend time working together to integrate all parts. You should also be maintaining active communication between group members to make sure you all have an understanding of how all your code works. Even if you did not work on a specific section, we expect you to be familiar with how the code works and be able to explain what it and how it fits into your kernel.

Throughout the first part of this semester and in most/all of your previous classes, MPs were more structured. You were given a set of functions to write, and you only modified those functions. One of the goals of this class is to make you into a more confident and thoughtful programmer by having you practice software design. We have deliberately left the implementation of certain sections open-ended. It is up to you and your group to find a way to meet the requirements - as with the real world, there is no “perfect” solution. The only requirement that we impose is that you follow the function interfaces that we have already specified - feel free create your own helper functions or creative implementations.

[†]Some may disagree that this is the most important thing, but who wants no games?

4 Testing

For this project, we strongly recommend that you write and run unit tests with adequate coverage of your source code.

As your operating system components are dependent on one another, you may find it useful to unit test each component individually to isolate any design or coding bugs.

You should create a `main_tests.c` file and add it to your `Makefile`. This file should create a kernel image (e.g., `test.elf`) that you can load into QEMU and run tests with. As you add more components to your operating system, we encourage you to add corresponding tests that verify the functionality of each component at the interface level.

Keep in mind that passing all of your unit tests does not guarantee bug free code. However, the test suite provides a convenient means to run your tests frequently without having to re-debug older components as you add new functionality.

5 What to Hand in

5.1 Checkpoint 1: Filesystem and Drivers and Program Loading, (Oh My!)

The primary motivation of this checkpoint is to get 2 test programs, which we've given you, to run. `hello` will print some text to the UART screen, then stop. `trek` will run the same text-based Star Trek game that you know and love.

Rather than do this in a “hacky” way, we want to set up some key infrastructure now which will pay dividends as the project continues on.

Note: Please ensure that all the functions that we specify do not induce a kernel panic on an invalid input. You should return an error code (which one is up to you) if possible, otherwise do nothing.

For the checkpoint, you must have the following accomplished:

5.1.1 Group Repository

You must have your code in the shared group repository, and each group member should be able to demonstrate that they can read and change the source code in the repository.

5.1.2 VIRTIO Block Device

An operating system in general must communicate with external devices. One such device is obviously the real drive/disk (virtual, in this case) which contains programs and other files you want your operating system to have access to.

In order to set up this device (and any others down the line), we will need to set up the necessary framework for the VIRTIO block device. Your group must finish the implementation based on the VIRTIO documentation linked on the course website.

You may find it especially helpful to read sections 2.1-2.7, 3.1, 4.2, and 5.2 and recall your implementation of `viornng.c`. Skip anything related to "legacy" interface.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

1. `void vioblk_attach(volatile struct virtio_mmio_regs * regs, int irqno)`
2. `int vioblk_open(struct io ** ioptr, void * aux)`
3. `void vioblk_close(struct io * io)`
4. `long vioblk_readat(struct io * io, unsigned long long pos, void * buf, long bufsz)`
5. `long vioblk_writeat(struct io * io, unsigned long long pos, const void * buf, long len)`
6. `vioblk_cntl(struct io * io, int cmd, void * arg)`
7. `vioblk_isr(int irqno, void * aux)`

See **Appendix B** for more information on the `io` struct.

5.1.3 Read-Only Filesystem

Broadly speaking, your filesystem driver should provide a comfortable interface to open, read and scan through files. In later checkpoints, you will add additional functionality.

Your `ktfs.c` file will need to interact with its backing device with some intermediate cache in `cache.c` to actually interact with the "physical" (well, virtual) device, so be sure that you understand what's going on in files related to both the cache and the backing device. Additionally, as this interacts with virtual devices, you should be sure that you use the `io` struct in the proper way.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

These functions should be written in `ktfs.c`:

1. `int ktfs_mount(struct io * io)`
2. `int ktfs_open(const char * name, struct io ** ioptr)`
3. `void ktfs_close(struct io * io)`
4. `long ktfs_readat(struct io * io, unsigned long long pos, void * buf, long len)`
5. `int ktfs_cntl(struct io * io, int cmd, void * arg)`
6. `int ktfs_flush(void)`

In order to use the filesystem, we have provided a `mkfs_ktfs` function (see **Appendix A**) that generates a filesystem image for you. This filesystem image is mounted by QEMU as a drive (using the `Makefile` we provide) and is accessible through `VIRTIO`.

For every “device” that uses `io`, you will need to implement “`ioctl`” `IOCTL_GETEND` and `IOCTL_GETBLKSZ`. Keep in mind that `IOCTL_GETBLKSZ` should not return the filesystem block size, but the “block size” of the file IO object - in this case, 1.

(Hint: implement the `memio` and its related functions to test your KTFS filesystem driver as well as the `cache` without the `vioblk`.)

See **Appendix A** for additional details.

5.1.4 Your Friendly Neighborhood Memio

You may wonder why we need a `memio` device if we already have a `vioblk` device? The `memio` device is a helper backing device that allows you to test your filesystem and ELF loader without using `vioblk`. Within your linker script `kernel.ld`, there is a section from `_king_blob_start` to `_king_blob_start` that you can use to place your whole filesystem image or an ELF file to load (see **Appendix A** about the blob).

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

For better unit testing and debugging, you must implement the `memio` device “driver” in `io.c`. The following is a list of functions you need to implement:

1. `struct io * create_memory_io(void * buf, size_t size)`
2. `int memio_cntl(struct io * io, int cmd, void * arg)`
3. `long memio_readat(struct io * io, unsigned long long pos, void * buf, long bufsz)`
4. `long memio_writeat(struct io * io, unsigned long long pos, const void * buf, long bufsz)`

5.1.5 Locked and ...

Locks prevent concurrency issues when multiple threads are accessing the same resource. You’ll need to implement the following functions and make modifications to your existing functions in `thread.c`. You should also modify your current thread struct so that it includes a member called `lock_list` that holds a linked list of all locks held by the thread.

Note: In order to prevent deadlocks, an exiting thread must release all held locks.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

You should implement the following functions in `thread.c`:

1. `void lock_init(struct lock * lock)`
2. `void lock_acquire(struct lock * lock)`
3. `void lock_release(struct lock * lock)`

5.1.6 ... (ELF) Loaded

One of the key roles of the operating system is to be able to run other programs.

We want to be able to run many user-level programs, but for now you can focus on `hello` and `trek`. While we've given you the pre-compiled binary of `trek`, you'll have to compile `hello` using the `usr/Makefile`. Because of this, you also have access to `hello.c`. All the binaries are in a format called ELF (Executable and Linkable Format), which has a specific layout — it is the standard for Unix and Unix-like systems, historically, which means it is still very relevant. See the **Tools, References, and Links** page on the course website for the Linux manual page on ELF. Your loader will only need to deal with the program headers, not sections, so focus on that documentation.

Notice that since `elf_load` should support any compliant I/O interface, that we can in general load an ELF from “any source” as long as `ioread` and `ioseek` are implemented in the given `io` (see **Appendix B**). (Hint: `memio`)

5.1.7 Cache Me Outside

“Software gets slower faster than hardware gets faster.” -Wirth’s Law

This semester, you will implement a caching system to cache blocks from a backing interface. As you've learned in this course, communicating with devices is extremely slow relative to the CPU's clock cycle. Previously, you've implemented asynchronous communication (condition variables) so that while one thread is waiting on a device response, another thread can run.

While this significantly reduces the problem of “wasting” CPU cycles, it does not eliminate the problem of latency - the original thread still must wait a long time for the device's response. A cache is a commonly used way to reduce this latency.

Once you complete this checkpoint, your backing interface will be the `VIRTIO` block device, but we will refer to it generically as the backing interface in this document. Rather than reading/writing a block directly from/to the backing interface, we first check whether the block exists in the cache. If the block exists in the cache, we access the block via the cache rather than sending a new request to the backing interface. If the block does not exist in the cache, we read it from the backing device into the cache. Note that you may or may not need to evict a block currently in the cache in order to bring in this new block.

Your cache may have any level of associativity and may be write-back, write-through, or some other concoction. Please note that we are intentionally leaving the specific details of the cache vague; this is intended to be a design exercise.

Some additional scenarios/specifications for the cache are as follows (these would be good test cases for you to write):

- Initially (when the OS is first started), the cache is empty.
- If the cache is initially empty and we read 64 contiguous blocks into it (e.g. 5, 6, 7, ..., 68), all 64 of those blocks must remain in the cache. This means that a read within any of those blocks should not generate a request to the backing device. (Hint: this should indicate to you that your cache capacity must be at least 64 blocks.)
- If `cache_get_block()` is called and block k is read into the cache, block k should remain in the cache at least until `cache_get_block()` is called again (note that block k may remain in the cache for longer). This is to say, if we call `cache_get_block()` once and do not call it again, the block that was cached must remain in the cache.

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

You will implement the following 4 functions in `cache.c`:

1. `int create_cache(struct io * bkgio, struct cache ** cptr)`
2. `int cache_get_block(struct cache * cache, unsigned long long pos, void ** pptr)`
3. `void cache_release_block(struct cache * cache, void * pblk, int dirty)`
4. `int cache_flush(struct cache * cache)`

As a reminder, you can also create any other helper functions that you need for your cache implementation.

5.1.8 Existing Files

During MP2, you worked with the PLIC, UART, and some other devices. You will be re-using that code for this checkpoint. You should add the following files into `sys` from MP2. They must be fully functional and (besides `thread.c`) should have the same functionality as a completed MP2. You can collaborate with your MP3 groupmates to choose whose MP2 code to use.

- `plic.c/.h`
- `rtc.c/.h`
- `uart.c/.h`
- `timer.c/.h`
- `thread.c/.h`
- `thrasm.s`
- `viorng.c`

5.1.9 Running Your Code

Finally, once you have all of your code completed, you will need to finish `sys/main.c` to run your program. We've left some comments on how to run `trek`, you may also find it helpful to refer to your MP2 CP3 `main` function implementation. You can also run `hello` or another user program that you create.

5.1.10 Troubleshooting/Debugging

See **Appendix H** for more information about debugging and common issues.

5.1.11 Checkpoint 1 Handin

For handin, your work must be completed and pushed to the `main` branch of your team's GitHub remote repository by the deadline.

For this checkpoint you must complete the following:

- Read-write operations on `vioblk`
- Read-only filesystem
- Read-write operations on `memio`
- Read-write operations on the cache
- ELF Load
- Locks

5.2 Checkpoint 2: Virtual Memory and Process Abstraction

5.2.1 Given Code

Linked on the course website are some extra files which are needed for the Checkpoint 2 implementation. Add them to your repo before starting Checkpoint 2 development.

Files to add to the `sys` directory:

1. `memory.c`
2. `memory.h`
3. `process.c`
4. `process.h`
5. `syscall.c`

6. `heap0.c` - This should replace your existing `heap0.c` and uses your virtual memory functions.

`trek_cp2` - This is a new binary for `trek` and uses U-mode and system calls. You should place it in your `usr/bin` directory.

5.2.2 Vicious Virtual Memory

In operating systems, paging is a memory management technique that allows non-contiguous allocation and efficient use of storage by dividing a process's address space into fixed-size units called pages. Page tables exist to maintain mappings between physical and virtual memory. When a process accesses a virtual address, the page table is queried to find the corresponding physical address. A page fault occurs when a process tries to access a virtual address that is not mapped to a physical address. If you handle a page fault that occurs within the User-owned virtual memory address space (`USER.START_VMA` to `USER.END_VMA`) you should allocate a new page and map that address as U-mode accessible. This is known as "lazy allocation" or "demand paging". For page faults that occur in other virtual memory regions, you should panic.

The operating system needs to support both single-page and multi-page allocations. When a process requires memory, the system selects the best-fit chunk from the free page list (the smallest chunk that fits the requested number of pages) to ensure optimal memory utilization. Pages can also be freed individually or in groups to ensure efficient reuse. When memory is no longer needed, in order to prevent security risks and memory leakage, the system must reset. This ensures clearing of non-global pages and freeing the associated memory.

Flags are used in memory management to control access permissions for maintaining security and track page usage. They are critical in allocating memory, mapping and unmapping of pages, validating memory access, as well as handling page faults. Access faults occur when a process tries to access a virtual memory address that they do not have permissions to access.

Note: This checkpoint will only contain a single memory space. You will not need to create new memory spaces for this checkpoint besides the "main" memory space.

We've given you a file, `memory.c`, where you will implement all the functions declared in `memory.h`. Keep in mind, many of these functions have overlap and it may be useful to look at the provided helper functions as well as write some of your own. You must write the following functions for this checkpoint:

In order to set up virtual to physical page mappings in your kernel, you must have a way to keep track of what physical pages are available to be mapped. To do this, we have created the "`free_chunk_list`". A `page_chunk` is a contiguous region of physical memory addresses. Each `page_chunk` contains a pointer to the next `page_chunk` in the list as well as a size (in pages). Initially, all of the memory from the end of the heap (`heap_end`, which is page-aligned) until the end of RAM (defined in `conf.h`) are free physical pages. You must modify `memory_init` to place all of these pages (in a single chunk) on the `free_chunk_list`.

Allocating and freeing physical pages must also interact with the `free_chunk_list`. To allocate physical page(s), you should go through the `free_chunk_list` and find a chunk that is greater than or equal the number of contiguous physical pages that you need to allocate. If there is no chunk large enough, you can panic. If there is a chunk, you should break off an appropriately-sized piece and provide a pointer to the start of the physical address range that was allocated. To free physical page(s), you can simply place the chunk back on the `free_chunk_list` - no need to coalesce chunks together.

Extra Credit Opportunity: Implement a more complex memory allocator (coalescing, buddy allocator,

etc.) and integrate it into your `kmalloc` function.

1. `void memory_init(void)`, most of this is given besides the initializing the free chunk list
2. `void reset_active_mspace(void)`
3. `mtag_t discard_active_mspace(void)`
4. `void * alloc_phys_page(void)`
5. `void * alloc_phys_pages(unsigned int cnt)`
6. `void free_phys_page(void * pp)`
7. `void free_phys_pages(void * pp, unsigned int cnt)`
8. `unsigned long free_phys_page_count(void)`
9. `void * map_page(uintptr_t vma, void * pp, int rwxug_flags)`
10. `void * map_range(uintptr_t vma, size_t size, void * pp, int rwxug_flags)`
11. `void * alloc_and_map_range(uintptr_t vma, size_t size, int rwxug_flags)`
12. `void set_range_flags(const void * vp, size_t size, int rwxug_flags)`
13. `void unmap_and_free_range(void * vp, size_t size)`
14. `int handle_umode_page_fault(struct trap_frame *tfr, uintptr_t vma)`

Some of these functions may build off of others. Some functions will also be called by functions used to implement processes (§5.2.3). Your code must meet the functionality requirements outlined in the rubric.

Consult **Appendix D** for more information about virtual memory.

5.2.3 Pesky Processes

The process abstraction is one of the key abstractions of an operating system. A process can be defined informally as just a "running user program". A user often wants to run multiple processes at once which requires common resources like processing power, devices, and memory to be managed.

An instance of a process structure contains everything that a process owns and uses internally. Each user process is actually just a wrapper around a kernel thread. What this means is whenever a user process is created, a process struct will have to be initialized to contain the information below:

1. An identifier for the current process
2. An identifier for the kernel thread related to this process
3. An identifier for the memory space of the process

4. A list of I/O interfaces for this process ; remember that an I/O interface can currently represent
 - A terminal device
 - An open file
 - A block device
 - An in-memory buffer

In this checkpoint, all user processes will share the same memory space, dubbed the "main" memory space. The execution lifecycle of a process will be as follows

1. The kernel launches in S-mode.
2. `procmgr_init` is called, creating a process struct around the main thread
3. `process_exec` jumps to user mode and starts executing a user program. Effectively turning the main kernel thread into a user process
4. When the user process exits, the kernel exits

Our kernel space process API is made up of the functions below. These functions will reside in `process.c` and should be written by you unless if stated otherwise:

1. `void procmgr_init(void)` (Provided)
 Initializes processes globally by initializing a process structure for the main user process (init). The init process should always be assigned process ID (PID) 0.
2. `int process_exec(struct io * exeio, int argc, char ** argv)`
 Executes a program referred to by the I/O interface passed in as an argument. We only require a maximum of 16 concurrent processes.
 Executing a loaded program with `process_exec` has 4 main requirements:
 - (a) First any virtual memory mappings belonging to other user processes should be unmapped.
 - (b) Then a fresh 2nd level (root) page table should be created and initialized with the default mappings for a user process. (This is not required for Checkpoint 2, as in Checkpoint 2 any user process will live in the "main" memory space.)
 - (c) Next the executable should be loaded from the I/O interface provided as an argument into the mapped pages. (Hint: `elf_load`)
 - (d) Finally, the thread associated with the process needs to be started in user-mode. (Hint: An assembly function in `trap.s` would be useful here)

Context switching was relatively trivial when both contexts were at the same privilege level (i.e. machine-mode to machine-mode switching or supervisor-mode to supervisor-mode switching), but now we need to switch from a more privileged mode (supervisor-mode) to less privileged mode (user-mode).

Doing so requires using clever tricks with supervisor-mode CSRs and supervisor-mode instructions. Here are some tips to consider while implementing a context switch from supervisor-mode to user-mode

- i. Consider instructions that can transition between lower-privilege modes and higher privilege modes. Can you repurpose them for context switching purposes?
- ii. If you did repurpose them for context switching purposes, what CSRs would you need to edit so that the transition would start the thread's start function in user-mode?

It's a useful exercise to try to figure out how such an approach could work with the CSRs and supervisor-mode instructions on your own. However, implementation on its own is a sufficient challenge and we don't require you to figure this out. You can read **Appendix C** to find out how you can carry out a context switch between user-mode to supervisor mode.

3. `void process_exit(void)`

Cleans up after a finished process by reclaiming the resources of the process. Anything that was associated with the process at initial execution should be released. This covers:

- Process memory space
- Open I/O interfaces
- Associated kernel thread

You will also have to modify your current threading library to accommodate for processes. To do this you will have to both declare the specified functions in your `thread.h` file as well as define them in your `thread.c` file.

The following functions will have to be implemented by you:

1. `struct process * thread_process(int tid)`
2. `struct process * running_thread_process(void)`
3. `void thread_set_process(int tid, struct process * proc)`

You also will have to add a `struct process * proc` to your `thread` struct

More information about the functions that you have to write can be found on the Doxygen site linked on the course website.

5.2.4 Suspicious Syscalls

You will need to implement a series of system calls (syscalls) for this checkpoint. The user program uses these to request actions from the kernel.

The syscalls you must implement for this checkpoint are:

1. `static int sysexit(void)`
2. `static int sysprint(const char *msg)`
3. `static int sysdevopen(int fd, const char *name, int instno);`

```

4. static int sysfsopen(int fd, const char *name)
5. static int sysclose(int fd)
6. static long sysread(int fd, void *buf, size_t bufsz)
7. static long syswrite(int fd, const void *buf, size_t len)
8. static int sysioctl(int fd, int cmd, void *arg)
9. static int sysexec(int fd, int argc, char ** argv)
10. static int syswait(int tid)
11. static int sysusleep(unsigned long us)
12. static int sysfscreate(const char * name)
13. static int sysfsdelete(const char * name )
14. void handle_syscall(struct trap_frame *tfr)
15. int64_t syscall(const struct trap_frame *tfr)

```

The function `handle_syscall` is used for system call linkage, through it calling the `syscall` function. Looking into the `syscall.S`, we see that system call exceptions are generating using the `ecall` instruction. The exception should be handled by `smode_trap_entry_from_umode` in `trap.s`. This assembly code should call the `handle_umode_exception` function in `excp.c`. User-mode exception handling is used for implementing both system calls and page fault handling. You will need to write both of these functions.

`smode_trap_entry_from_umode` will be the entry point to the kernel when a trap is generated in U-mode. Similar to it's S-mode equivalent, you must save registers to the trap frame, go to either the exception handler or interrupt handler, restore registers, and return. However, there are subtle differences that you will need to reason about and understand to make this function work. We highly recommend that you thoroughly read and understand `smode_trap_entry_from_smode` before attempting this function.

`handle_umode_exception` will be similar to `handle_smode_exception`, but keep in mind that we “gracefully” handle 2 exception types: page faults and environment calls. If one of these occurs (and is handled without an error) you should return from this function through `smode_trap_entry_from_umode`, otherwise, you should exit the current process.

5.2.5 Fully Implemented Filesystem

In Checkpoint 1, you already implemented a read-only KTFS driver. In this checkpoint, you will extend your filesystem driver to allow for writing to files as well as file creation and deletion. Keep in mind that all previous read behavior should be maintained, and after this checkpoint, your filesystem should implement CRUD functionality with respect to files.

Writes to the disk should persist within the filesystem image even after QEMU shutdown. That is, if you run a program that creates a file and writes to it (assuming it is written to the `vioblk` device through the cache),

then shut down QEMU, you should be able to open and read from that file the next time QEMU is launched. This also means that you can look at your filesystem image file after QEMU exits to help debug writes. Be careful that the writes are truly persisting to disk - if you have a write-back cache, they may only be written to the cache before QEMU exits.

Note: Writing to files is a bit trickier than reading from them, but there is a lot of overlap code. Try your best to follow DRY (Don't Repeat Yourself).

For this checkpoint, you should implement the following functions in `ktfs.c`:

1. `long ktfs_writeat(struct io* io, unsigned long long pos, const void * buf, long len)`
2. `int ktfs_create(const char* name)`
3. `int ktfs_delete(const char* name)`

You must also implement `IOCTL_SETEND` in `ktfs_cntl`. `IOCTL_SETEND` should extend the length of the file and add additional data blocks if needed.

You should add `fscreate` and `fsdelete` functions with the appropriate signatures to `fs.h` and alias them to your KTFS driver functions (see `ktfs.c`).

It is very important that you write thorough tests for the filesystem. There are many possible edge and corner cases and we highly encourage you to create targeted tests to ensure that you handle all of them properly.

5.2.6 Additional Modifications

Due to the new virtual memory, process abstraction, and system calls, we are now able to run programs in User-mode instead of Supervisor-mode. This means that some of the code you wrote in Checkpoint 1 will no longer work. Here is a (non-exhaustive) list of modifications you'll have to make to your existing code:

1. `elf.c` must be modified to use virtual memory. User programs should now be loaded between the `0x0C0000000UL` and `0x100000000UL` virtual addresses. If you used the provided `USER_START_VMA` and `USER_END_VMA` constants in `conf.h`, you should update them to these values as well. You will need to allocate and map the appropriate amount of pages, as well as set the appropriate flags in the page table when mapping the program segments.
2. `main.c` should be modified to initialize memory and the process manager. You should also remove `heap_init` from `main.c`, as `memory_init` will do that for you.
3. `trap.h` must be modified. You should change the `trap_frame_jump` function declaration from

```
extern void __attribute__((noreturn)) trap_frame_jump(struct trap_frame * tfr)
```

to

```
extern void trap_frame_jump(struct trap_frame * tfr, void* sscratch) __attribute__((noreturn))
```
4. `sys/Makefile` needs to be modified to due to the new files that you added.

5. `usr/Makefile` needs to be modified to support the new User-mode program address space. We have already provided a new linker file (`umode.ld`), so you only need to change the `UMODE` flag in `usr/Makefile` to 1. **Note: Once you make this change, you must recompile any user programs.**
6. `scnum.h` and `usr/scnum.h` must be modified to add the create and delete syscall numbers. Add `#define SYSCALL_FSCREATE 12` and `#define SYSCALL_FSDELETE 13` to both of these files. You will also need to modify `usr/syscall.S` to add U-mode support for these two syscalls. To do this, create 2 new functions `_fscreate` and `_fsdelete`. They will be very similar to other syscall functions in the file.
7. In order to make `io` devices able to use the read/write system calls, you must implement `seekio` wrapping in the “open” functions. To clarify, this means that any IO devices that are intended to be used in U-mode via syscalls (namely files) must return a seekable IO struct. This is due to the fact that we only have `sysread/syswrite` system calls, and not `sysreadat/syswriteat`. To fix this, the “open” functions (in this case, `ktfs_open`) must return a `seekio`-wrapped IO pointer.

5.2.7 Checkpoint 2 Handin

For handin, your work must be completed and pushed to the `main` branch of your team’s GitHub remote repository by the deadline.

For this checkpoint you must complete the following:

- Reading, writing, creation, and deletion of files within KTFS
- Correct virtual memory mappings
- Correct virtual memory functionality
- Process abstraction
- System calls from U-mode

We have given you a new binary, `trek_cp2`, which uses *some but not all* of this functionality. While being able to run this program is a good sign, it does not necessarily mean that all functions are correct and you are handling edgecases properly. You should write your own testcases to verify functionality.

5.3 Checkpoint 3: Forks, Preemptive Multitasking, fd Duplication, Pipes, Validation and Shell

5.3.1 Friendly Forking

“Fork this, I’m out” -KIP

We **heavily** recommend to finish this first as `shell` is dependent on `forking`.

In Checkpoint 3 we need to support “forking” processes so that we can have multiple processes running in our system. We’re starting off with the `fork` syscall which has the following function signature.

- `static int sysfork(const struct trap_frame * tfr)`

To implement the `fork` system call you will have to fill out the following functions in `process.c` which are described in the doxygen.

- `int process_fork(const struct trap_frame * tfr)`
- `void fork_func(struct condition * done, struct trap_frame * tfr)`

You will also have to declare the following function in `memory.h` and implement it in `memory.c` to use in `process_fork`. It is described in the doxygen.

- `mtag_t clone_active_mspace(void)`

Another thing to note is you will now have to deal with multiple memory spaces. This means that you have to switch between them when switching between processes. This change will have to be done in `thread.c`. You should also change the `kmalloc` in `create_thread` when allocating a stack page to a `alloc_phys_page` so that each new process gets a full page for their kernel stack. You will also have to change the corresponding `kfree` in `running_thread_suspend` to a `free_phys_page`.

You may be delighted to hear that the `fork` implementation here is virtually identical to how it’s done in “real” UNIX-like operating systems, yay!

5.3.2 Perniciously Meticulous Preemptive Multitasking

In this checkpoint you will be dealing with preemptive multitasking for programs running in user mode. Preemptive multitasking is very similar to the cooperative multitasking from MP2 but instead of calling `thread_yield` synchronously through the program itself during execution, it is called asynchronously via the interrupt handler. This will preempt the currently running thread by causing it to suspend and hand over control to the next waiting thread. Remember, we will only preempt the thread if we are running programs in the user mode. This can be seen in `handle_umode_interrupt`. This will allow switching between kernel threads (and therefore user processes) when `forking` creates additional ones (besides `main`).

5.3.3 Devious Duplication

It is often useful for processes to have multiple file descriptors reference the same I/O object under the hood. However, we don't allow users to open the same device or file multiple times. To solve this, you must implement a new syscall `sysiodup`, referred to by user programs with `iodup`. This syscall takes in two file descriptors and copies the I/O device from one to the other. Remember to update your the refcnt! `sysiodup` will be associated with the syscall number 21. You must add this to `senum.h` in both the `sys` and `usr` directories. The function is shown below and is described in the doxygen, it should be written in `syscall.c`. Don't forget to modify `syscall.S`.

- `int sysiodup(int oldfd, int newfd)`

5.3.4 Pervasive Pipes

In this checkpoint, you will be adding support for UNIX-style one-way pipes in your operating system. You will create an I/O interface provider that would function similarly to your existing I/O interfaces and will include the logic required for sending data between different processes.

You will need to create two different I/O interfaces for this purpose, one for the writer-end and one for the reader-end of the pipe. Both will reference the same buffer, allowing the writer process to send data to the reader process. Further details can be found on the doxygen.

You will also have to define the following in `io.h` and implement it in `io.c`, further detail can be found on the doxygen

- `extern void create_pipe(struct io ** wioptr, struct io ** riopttr)`

You will have to write the full pipe implementation in `io.c` in order to make your pipes implementation work. You may have to create helper functions as well as functions to populate your IO interface.

Finally, you will need to update your syscalls to support user process piping. To do this, you will need to add an additional syscall `syspipe` associated with syscall number 20. Its function signature is as follows and more detail can be found on the doxygen. As always when adding system calls you will have to modify `syscall.S`.

- `int syspipe(int * wfdptr, int * rfdptr)`

5.3.5 User Input Validation Extra Credit

When accepting user input the kernel should always validate the inputs from the program. This is especially important with memory as accessing invalid memory locations can cause the kernel to crash. To prevent cases like this for this checkpoint you are going to implement the following functions which will validate user input in syscalls.

You should declare the following functions in `memory.h` and define them in `memory.c`. Follow the doxygen for any confusion regarding the declarations. These functions are for EXTRA CREDIT and not required for MP3.

- `int validate_vptr(const void * vp, size_t len, int rwxu_flags)`
- `int validate_vstr(const char * vs, int ug_flags)`

To receive any points for these functions, they **MUST** be used **WHENEVER** processing input from the userspace (AKA in syscalls). On failure of a validation check you must return an error.

5.3.6 Welcome to (S)hell

At this point, you have implemented everything you need to have a fully functional Operating System! To test your full range of capabilities, we have provided you with a basic shell that runs on top of your kernel. Add this binary to your filesystem and modify `main.c` to execute this program after your initial setup. The shell takes in no arguments but does require a console device in file descriptor 2.

Once you are able to enter text into your UART device, your shell is ready to go. To run any executable in your filesystem simply type it into your shell (> is not part of what you should input).

```
> trek
```

This will `fork` and have the child run `_exec(trek_fd, 0, NULL)` while the parent waits for the child to complete. However, unlike in Checkpoint 2, you should return to your shell upon exiting the game.

You can also specify the programs arguments, where subsequent values will be passed into the executable's `argv`.

```
> rogue rogue savedata
```

This command will be run as `_exec(rogue_fd, 2, {"rogue", "savedata"})`. Note that the shell does not support double quotes ("") and they will not be removed from the string.

In general, user programs will perform all reads from `stdin` (file descriptor 0) and all writes to `stdout` (file descriptor 1). By default, `stdin` is empty, and `stdout` is the console. However, you can change this via shell arguments.

```
> proc<file
```

Is the same as first running `_fsopen(0, "file")` followed by `_exec(proc_fd, 0, NULL)`. Writing to a file works in a similar fashion, with `file` being opened in file descriptor 1.

```
> proc>file
```

If `file` is not in your filesystem, however, it will be created with `_fscreate` in this case.

The final feature this shell supports is piping. You can set up two programs to communicate with each other, setting a pipe to be one program's `stdin` and another program's `stdout`.

```
> proc1|proc2
```

This command will pipe to get the file descriptors, create two processes to run concurrently, and finally execute both programs on each process.

You'll likely notice that the provided shell has quite a lot of limitations. You are welcome to and in fact encouraged to create your own, better shell from scratch, especially to demonstrate any extra credit features

you implement. Keep in mind that for this checkpoint, however, you will be graded primarily through the provided shell, so it is expected to fully function as specified.

You should write your `main.c` such that it will run the provided shell on startup.

5.3.7 Checkpoint 3 Handin

For this checkpoint, you should be able to show that all the core functionality required here works. Namely:

- Be able to run shell, your `main.c` should run shell by default
- Be able support forking and pipes
- Ability to run multiple processes concurrently
- Be able to compile all the files associated with your kernel and user programs.

6 Grading

Your final MP score is a baseline score that will be adjusted for teamwork and for individual effort. Note that the “correct” behavior of certain routines includes interfaces that allow one to prevent buffer overflows (that is, the interfaces do not leave the size of a buffer as an unknown) and other such behavior. While the TAs will probably not have time to sift through all of your code in detail, they will read parts of it and look at your overall design. The rough breakdown of how points will be distributed will be periodically released on the website prior to checkpoint deadlines.

A Note on Teamwork

Teamwork is an important part of this class and will be used in the grading of this MP. We expect that you will work to operate effectively as a team, leveraging each member’s strengths and making sure that everyone understands how the system operates to the extent that they can explain it. In the final demo, for example, we will ask each of the team members questions and expect that they will be able to answer at a reasonable level **without** referring to another team member. **Failure to operate as a team will significantly reduce your overall grade for this MP.**

At the end of the MP there will be a teammate evaluation which will factor into your final MP grade.

You are also expected to adhere to the MP3 contract.

7 Appendix A: The File System

7.1 File System Overview

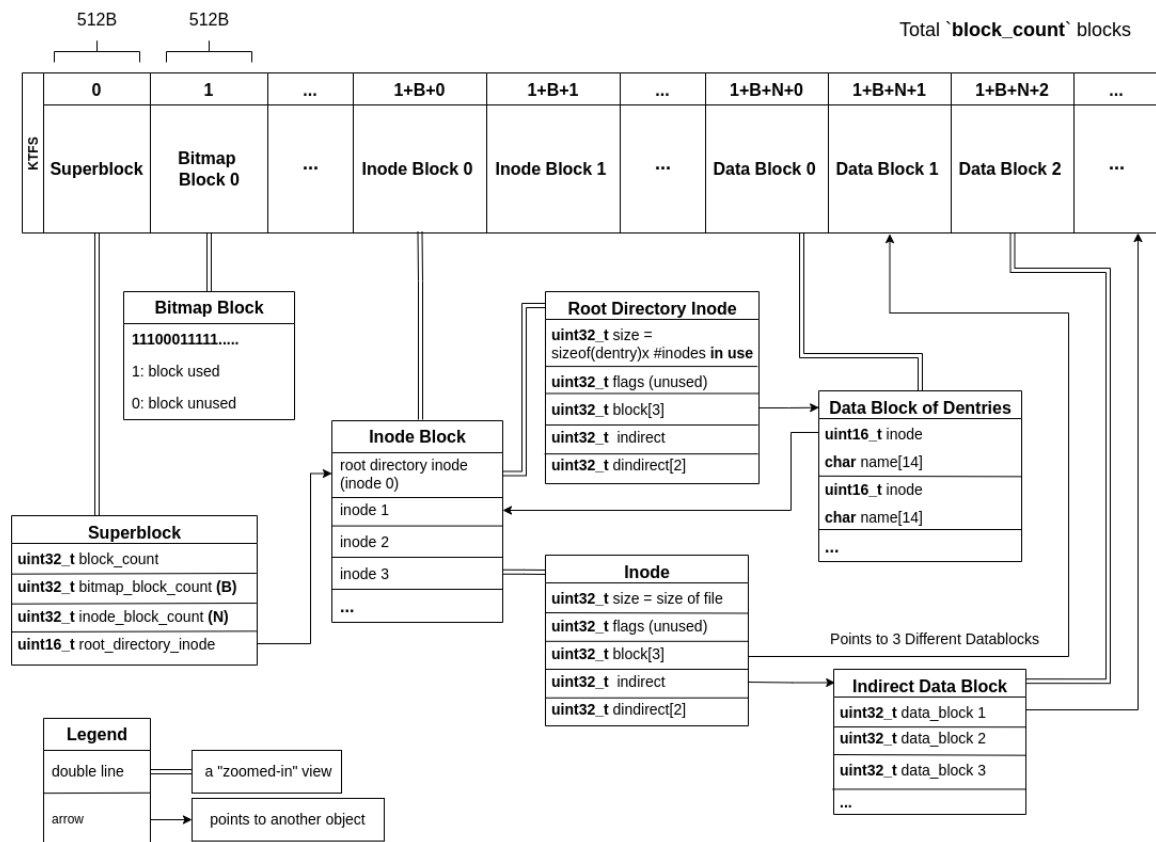
A filesystem can be considered an abstraction that allows for easy access of one or more "files" within a large chunk of data. For this MP, you will be implementing KTFS - a custom filesystem inspired by ext2.

First, we will provide definitions of important terms.

The *filesystem image* refers to a "blob" of data that contains everything needed to mount the filesystem and all the data associated with it.

The *backing device* for a filesystem refers to where the filesystem image is stored. In an actual computer, it's typically stored on the hard drive. However, for this MP, the backing device will be the VIRTIO block device, which emulates a real hard drive. To test your filesystem without the VIRTIO block device, you can use a buffer in memory (see `memio`).

The figure below shows the structure and contents of the filesystem image.



Layout of KTFS

KTFS is a block-based filesystem. We use a block size of 512B, which coincidentally matches with the block size of the VIRTIO device. There are 4 types of blocks:

1. Superblock (stores basic statistics of the file system image)
2. Bitmap blocks (keeps track of which blocks are in-use)
3. Inode blocks (stores index nodes (inodes), which contain info about each file)
4. Data blocks (stores actual data)

The first block is called the **superblock**. The superblock has four fields:

- `uint32_t block_count` - the total number of block in the file system
- `uint32_t bitmap_block_count` - the number of bitmap blocks
- `uint32_t inode_block_count` - the number of inode blocks
- `uint16_t root_directory_inode` - the index of the root directory inode (see below)

These fields only take up the first 14B of the 512B, so the remaining bytes are unused. The fields in the superblock are set when creating the filesystem image (see §**Building the File System**) and should not be modified by your OS.

The blocks following the superblock are the **bitmap blocks**. The bitmap blocks contain "bits", as the name suggests. Each bit indicates whether a block is used: 1 means used, 0 means free. For example, the 0th bit of the 0th bitmap block corresponds to the 0th block of the filesystem. If a block is free, then it can be used later when writing to a file or creating a new file. When the filesystem image is created, the superblock, bitmap blocks, and inode blocks are all marked as in-use. During filesystem image creation, the "disk size" is set and fixed, which means the number of bitmap blocks is also fixed.

The **inode blocks** contain the index nodes (inodes) of the files in the filesystem. Each file is described by an inode that specifies the file's size in bytes and the numbers of the data block (e.g. data blocks 1,5,10 ...) that make up the file. During filesystem image creation, the number of inode blocks is set and cannot change.

Each inode only provide direct access to 3 data blocks through an array within the inode. However, inodes additionally store 1 indirect data block number and 2 doubly-indirect data block numbers. An indirect data block number points to a data block that contains an array of data block numbers, instead of actual data. For example, if you want to access the 4th data block of a file, you will have to get the indirect data block number first (e.g. `inode.indirect = 11`), get data block 11 from backing device, and read the first `uint32_t` (first 4B) of data block 11 (e.g. 14). Then, go to data block 14 and there will be actual data. The idea is the same for doubly-indirect blocks, except there's one more layer of indirection: the doubly-indirect data block number points to a data block that stores an array of indirect data block numbers, which further points to indirect data blocks that store an array of "real" data block numbers.

Each indirect/doubly-indirect data block contains 512B (just like any other data block), but only the part of the indirect/doubly-indirect data block that points to data blocks necessary to contain the specified size need be valid, so be careful not to read and make use of block numbers that lie beyond those necessary to contain the file data. The data blocks that make up a file are not necessarily contiguous or in any specific order.

You must use the data block numbers in the order specified in the inode and the indirect/doubly-indirect data blocks to access the correct data in the correct order.

There's one special inode called the root directory inode. Instead of containing the actual data of a file in its specified data blocks, these data blocks contain the directory entries. KTFS does not support nested directories, so the root directory is the only directory that you are required to support. Each directory dentry (or dentries for short) contains an inode number and a string that contains the file name. The size of each dentry is 16B: 2B for the inode number and 14B for the file name. Note that because we need 1B for the null terminator, the maximum length of a file name is 13B (13 characters). The "file size" specified in the root directory inode is equal to (the number of dentries) $\times$ (the size of each dentry). Therefore, you can determine the number of files in the filesystem by checking the size field of the root directory inode. When you create or delete a file, you must add/remove the dentry for that file and update the size appropriately. For this file system, there is a 1:1:1 correspondence between files, dentries, and inodes. No two dentries should contain the same inode number or file name.

Note: Dentries **must** be contiguous to be able to use the file size properly. This is a strict requirement that filesystem images will follow and your filesystem driver must maintain. Keep in mind that "contiguous" here does not actually mean contiguous in memory, just that if you were reading the root directory inode like a file, the first `size` bytes would be the dentries.

In most cases, the **data blocks** can be seen as normal chunks of data in a file. That means when you open a file, you should be reading data from and writing data to these actual data blocks. However, because of the root directory inode and indirect data blocks, data blocks can contain different things. So, a data block can be classified into the following types depending on what data it contains:

1. "Real" data blocks (contains actual data of a file)
2. Directory data blocks (pointed to by the root directory inode, contains directory entries)
3. Indirect data blocks (contains an array of data block numbers)
4. Doubly-indirect data blocks (contain an array of indirect data block numbers)

Note: Since the root directory inode is treated similar to a "regular" inode, it can grow in size like other files with indirect and doubly-indirect data blocks. This means that the limiting factor on the number of files in a filesystem is typically the number of inodes.

7.2 File Abstractions

In order to make your filesystem driver, you should create a `struct ktfs_file`. This will be the internal representation of a file that your filesystem driver uses to keep track of the state of each file. This structure should contain *at least* the following fields (add more if you find it useful):

1. The `io` struct associated with each file. The `intf` field in this structure should be a pointer to an `io_intf` struct that contains the correct `close`, `readat`, `writeln`, `cntl` function pointers to interface with the file.

2. A **file size** member that stores the length of the file in bytes. This should be updated whenever write/writeat is called, if the length of the file has changed.
3. The **directory entry** struct for this file. As a reminder, this contains the inode number for the file and the file name.
4. A **flags** member for, among other things, marking this file struct as “in-use.”

You need to have some way to keep track of the `ktfs_file` structs that correspond to currently opened files (array, linked list, etc.), so that future I/O operations can be performed on them.

7.3 Building the File System

The filesystem image needs to be made in order for QEMU to read it and make it accessible to you (via VIRTIO). To do this, we have provided you with 2 binary executables within the `util/fs` directory.

- `mkfs_ktfs` is a customized version of Linux `mkfs` that generates a filesystem image according to the KTFS spec detailed above. This is how we will generate filesystem images to test and grade your filesystem driver.
- `mkfs_ktfs_inorder` is a modified version of `mkfs_ktfs` that does not randomize the order of data blocks associated with each file. **This is provided for debugging purposes only** and will not be used to grade your filesystem driver. You must be able to interface with files that use non-contiguous data blocks.

We have also provided you with a 3rd binary executable, `unmkfs_ktfs`. This program “unwraps” the files contained with a KTFS filesystem image to allow you to parse them on your own. You may find this useful when implementing the write/writeat functions in your filesystem driver, since these functions will modify your actual filesystem image and persist between QEMU shutdowns.

`mkfs_ktfs` usage is as follows:

```
./mkfs_ktfs [filesystem_image]
            [disk_size in K, M, or G]
            [inode_count]
            [file1] [file2] ...
```

In order for your filesystem image to work with our existing `Makefile`, you should name the image `ktfs.raw` and put it in the `sys` directory.

An example:

```
./mkfs_ktfs ../../sys/ktfs.raw 8M 16 ../../usr/bin/hello ../../usr/bin/trek
```

This would create a `ktfs.raw` file in the `sys` directory with a total size of 8MiB and 16 inodes (*i.e.*, 1 inode block). `hello` and `trek` would be the only 2 files in the filesystem image, with the rest of the inodes free.

Since we cannot have fractional data blocks, your disk size should always be a multiple of 512 (KTFS block size). The number of inode blocks is also rounded up, since we cannot have fractional inode blocks. When interfacing with the filesystem, you can consider any block marked as an "inode block" as able to be used to store inodes. In other words, the total number of possible inodes in your file system will always be a multiple of 16. However, bitmap blocks may be "partially" used. Be careful to not read/write past the size of the filesystem image.

Note: Make sure you are running your `mkfs_ktfs` binary and not the Linux `mkfs` function.

The files in your filesystem image are also given as arguments to `mkfs_ktfs`. You can (and should) provide multiple files in order to create a full filesystem image. All the metadata related to these files will be prepared for you, including the superblock and other structures that you've read about in **Appendix A** already. All of these blocks will be put in order according to the spec above and stored in your filesystem image (*i.e.*, `ktfs.raw`). This means that the filesystem image is already compliant with the KTFS spec.

To load a user program into the filesystem image, you need to compile it to an executable binary. This can be done by using the `Makefile` in `usr`. If you want to change a file in the filesystem or add/remove files (without using your filesystem driver), you **must** remake the filesystem image each time.

7.4 The Blob

The blob will be created as an object file and is not part of the regular filesystem. It is a tool for debugging and testing. If `blob.raw` exists in your `sys` directory, the `Makefile` will input the file contents of `blob.raw` to a read-only data section starting from `_kimg_blob_start` to `_kimg_blob_end`. These addresses can be obtained via `extern`. If you want to use the blob, simply rename the relevant file to `blob.raw` in the `sys` directory and add the object file when you build your kernel image ELF file as well as `extern` both addresses in your code to access the blob.

8 Appendix B: I/O Devices

8.1 Overview

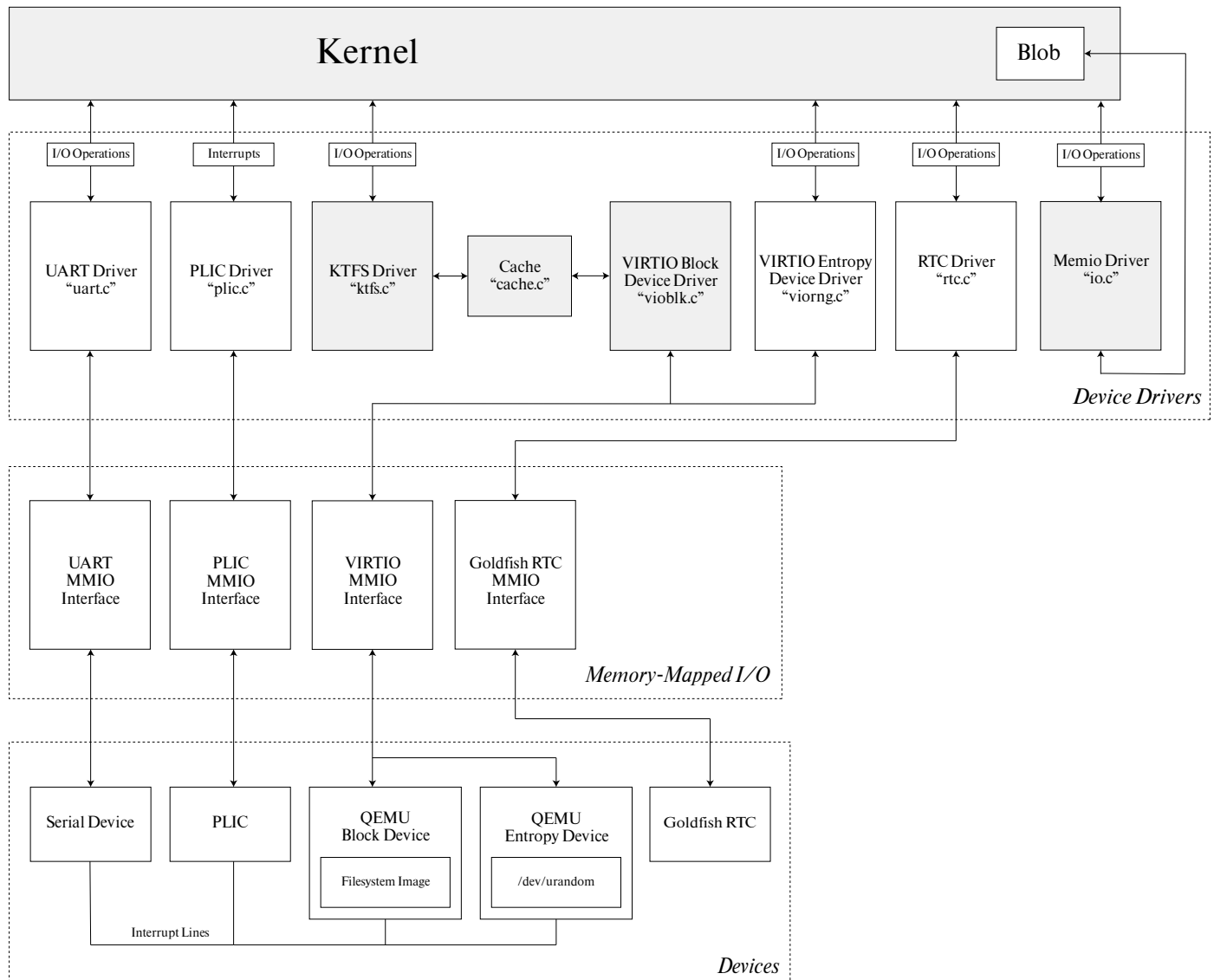
For Checkpoint 1, you must support a variety of devices, most of which use the `io` abstraction. When interacting with devices, abstraction is necessary. Historically, companies have come together in consortiums related to networking and inter-device communication (see PCI-E, USB, CCIX). These interconnects enable fast, standardized communication between devices.

QEMU is simulating devices and their interconnects. Your filesystem image, for example, (see **Appendix A**) will be managed by the QEMU block device.

8.2 I/O Operations

The kernel will interact with devices using the struct `io`. Each struct `io` contains a pointer to a struct `io_intf`, which in turn contains pointers to `close`, `cntl`, `read`, `write`, `readat`, and `writeln` functions specific to each device driver. Note that not all these functions need to be implemented - if the device driver does not implement the function, the pointer can be `NULL`. Under the hood, these callback functions to the device drivers will use the memory-mapped I/O regions to communicate with devices.

8.3 I/O Diagram



The shaded blocks are the parts that you will write as part of MP3.

9 Appendix C: The Process Abstraction

9.1 History

The process abstraction in Unix like systems is a critical part of understanding an operating system's inner workings. Historically, processes were the first abstraction over a user program rather than threads. User programs were seen as having a single flow of instruction and resided in separate memory spaces. As computer hardware advanced and provided stronger computing, it became desirable to have multiple streams of instructions (threads) running within a single address space. For Checkpoint 2 we will only consider processes that contain only a single thread.

9.2 Overview

The Linux kernel's smallest abstraction over a stream of instructions is a thread. For a UNIX-like system (like ours), each process by default will be spawned around a main thread. This main thread is what is scheduled by the operating system.

A process being scheduled by the kernel to run is just the process's thread being put on the ready list and a process running is just the thread associated with the process running (as the current thread).

On top of owning a thread, each process also owns a virtual memory space. This virtual memory space is represented by a 3-level page table. You can read more about the paging and virtual memory in **Appendix D**.

In addition, a process should also keep track of files and devices that it's interacting with. Note that we have a common interface for files and devices (`io`)

The process abstraction provides 3 key guarantees to a user process:

- **Protection:** Assures that a malicious process cannot corrupt, fault or deny other user processes.
- **Virtualization:** Creates the illusion to a process that resources are boundless and only used by that process, *i.e.* virtual memory, illusion of concurrency with scheduling, etc.
- **Abstraction:** Abstracts away implementation details and provides a common interface to user processes, *e.g.* a user program opening a file is the same code regardless of the file system being backed by an `memio` or a `vioblk` device. They are covered by the same system call.

9.3 Context Switching between User-Mode and Supervisor-Mode

Context switching between two privilege levels fundamentally requires two instructions with two unique properties:

1. An instruction to go from a lower privilege level to a higher privilege level
2. An instruction to go from a higher privilege level to a lower privilege level

The ECALL Instruction:

The environment call (`ecall`) instruction will be our way of going from a lower privilege level (user-mode) to a higher privilege level (supervisor-mode). It is meant to be used by user-mode software to make a service request to a higher privilege context.

Reading the description for the `ecall` instruction in the RISC-V Privileged ISA manual we can see that it will trigger an exception which should be delegated to the supervisor-mode trap handler. This exception being triggered from user-mode will have the following effects:

1. The `sepc` register is loaded with the virtual address of the instruction that triggered the exception (*i.e.* the `ecall` instruction itself).
2. The `scause` register contains the cause code for the exception (the cause code for an environment call from user-mode).
3. The `sstatus` register's Supervisor Previous Privilege bits (SPP) will be set to 0 to indicate that it is a trap from user-mode.
4. The `sstatus` register's Supervisor Previous Interrupt Enable bits (SPIE) will indicate whether or not supervisor-mode interrupts were enabled prior to trapping into supervisor-mode.
5. The `sstatus` register's Supervisor Interrupt Enable bits (SIE) will be cleared to disable supervisor-mode interrupts while in supervisor-mode.

The SRET Instruction:

The supervisor return (`sret`) instruction is what we'll use to transition from a higher-privilege level (supervisor-mode) to a lower privilege level (user-mode). It is meant to return to user-mode once a user request has been completed.

Reading the description for the `sret` instruction in the RISC-V Privileged ISA manual we can see that an `sret` in supervisor-mode will return to a lower privileged mode (user-mode) with the following effects:

1. The `sstatus` register's SIE bit will be set to SPIE.
2. The `sstatus` register's SPIE bit will be set to 1.
3. The `sstatus` register's SPP bits will determine the execution privilege mode after `sret`.
4. The `sstatus` register's SPP bits will be set to the least-privileged supported mode (user-mode).
5. The `pc` register is set with the value of `sepc`.

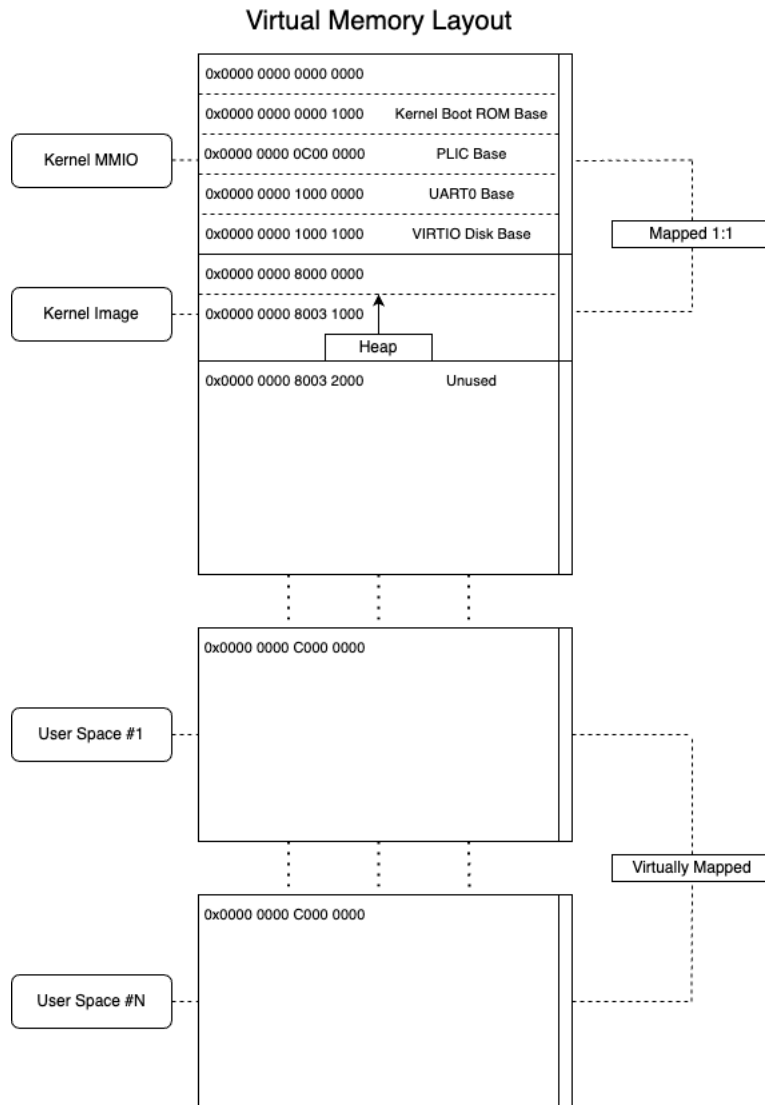
Now that we know the effects of `ecall` and `sret`, we can reason about how to use these instructions during context switches. There are two specific cases to consider:

- **Switching to supervisor-mode from user-mode:** This type of switch is the most straightforward transition and does not require any preparation. You can just use the `ecall` instruction.

- **Switching to user-mode from supervisor-mode:** Since our kernel initially boots into supervisor-mode, it can be a bit tricky to figure out how to use `sret` to transition into user-mode. This confusion may come up because at first glance `sret` requires a prior user program to have trapped into supervisor-mode (so that the SPP and SPIE bits will be pre-populated with values). However, if this is the first time a user program is being executed there can't be any prior user-mode trap frame and SPP and SPIE cannot be filled in. It is up to the `process_exec` function to fill in the proper values for SPP and SPIE so that `sret` can properly jump to a user-space function.

10 Appendix D: RISC-V Paging

10.1 Illinix Mappings



10.2 Sv39

RISC-V offers a few different paging schemes, each of which offers a different virtual memory size. We are using Sv39 for this MP. Each `struct pte_t` (defined in `memory.c`) describes a page table entry specific to Sv39. You will need to look at the RISC-V privileged docs to learn more about each field: [here](#).

Virtual memory address translation, also called `paging`, is a hardware-level tool that supports process virtualization. When paging is enabled and the machine is in `umode`, the hardware will translate all user-program memory accesses from virtual to physical addresses.

Multiple programs may be staged to execute at the same address. Memory virtualization allows each process to execute with the idea that they have the entire memory space at their disposal, and ensures that the operating system can manage many programs executing in parallel.

To use the QEMU console (useful in debugging) you can add the following line in `src/kern/Makefile` where you set your `QEMUOPTS`

```
QEMUOPTS += -monitor pty
```

When running `make run-kernel`, you will see another `/dev/pts/N` that you will need to connect to in order to interact with your QEMU console. To view the virtual memory mappings of your system, you can run `info mem` in your qemu console.

11 Appendix H: Troubleshooting

11.1 Debugging with gdb

“If debugging is the process of removing bugs, then programming must be the process of putting them in.”
-Edsger W. Dijkstra

One of the most important skills you can have as a software engineer is being able to debug. While print statements are useful and easy to implement, using a real debugger like gdb will allow you to find and solve your problems much more quickly. If you invest time into learning it at the start of this MP, it will pay dividends for the rest of the assignment and future classes. Many ECE 391 alumni (including course staff) have said that the most important thing they learned from the course was not how to make an operating system, but how to debug effectively on their own. While course staff are there to help you, they are not there to handhold you and it is expected that you are able to debug on your own, especially with gdb.

11.1.1 Debugging the Kernel

This section describes how to set up your kernel to work with gdb. This will allow you to step through kernel and user program code if it is compiled with debugging symbols (`-g`).

Whenever a change is made to your kernel, you should rebuild your kernel using `make`. For the purposes of this appendix, we’ll be describing debugging with the `kernel.elf` kernel image, which we’ve given you. If you’re using a different kernel image, use that instead. You can create different kernel images by writing an appropriate `main.<prog>.c` file and adding it to the `Makefile`.

To launch QEMU and have it wait for remote debugging, you must include the `-s` flag, which we have done for the command `make debug`. This will launch your `kernel.elf` kernel image as normal, but no instructions will execute until you connect to it with gdb and run `continue`.

Once you’ve launched your kernel, you need to open a second terminal and run

```
riscv64-unknown-elf-gdb [kernel image]
```

This will run the correct version of gdb and load the debugging symbols from your kernel image, allowing you to set breakpoints and step through the code. We have included a `.gdbinit` file in the `sys` directory, which executes a few commands on gdb startup — this should cause it to automatically connect to your QEMU instance. To use the `.gdbinit` file, simply launch gdb from the `sys` directory.

Note: The first time you use a `.gdbinit` file, you might get a message that looks like this

```
warning: File "/path/to/src/kern/.gdbinit" auto-loading has been declined by
your 'auto-load safe-path'.
To enable execution of this file add
add-auto-load-safe-path /path/to/src/kern/.gdbinit
line to your configuration file "~/.config/gdb/gdbinit".
```

In order to resolve this, you should simply follow the instructions that gdb gives you and add the path. You may have to use `mkdir` to make the `~/.config/gdb` directory before using your favorite text editor to add

the `add-auto-load-safe-path /path/to/src/kern/.gdbinit` line into `~/.config/gdb/gdbinit`. You only need to perform this setup process once per `.gdbinit` file, even if you modify it in the future. It is **highly recommended** that you use the `.gdbinit` file to save you the trouble of having to connect to the QEMU instance every time.

Note: You may need to modify the `target remote 127.0.0.1:1234` line in your `.gdbinit` file to match the port your QEMU instance uses. The `-s` flag provided to QEMU allows it to use the default port (1234), but if you are having trouble connecting with that port, you can explicitly specify which port to use in the QEMU command. Then, modify your `.gdbinit` file to match.

11.1.2 Debugging User Programs

If you want to set breakpoints or step through a user program, things are slightly more complicated. Since `kernel.elf` (or whatever your kernel image is) does not contain the user program, we need to add the debugging symbols for it separately into `gdb`.

First, you need to compile your user program with debugging symbols enabled. In the `Makefile` we have provided, we have already enabled debugging symbols (`-ggdb3`). If you write your own user program, you can add it to the `Makefile`. To find the address of the program itself, run

```
readelf -Wl [program binary]
```

This gives you information on where different sections of your program are loaded. Find the executable section (E flag) and note down the `VirtAddr`. For checkpoint 1, it should be located around `0x80100000`. For later checkpoints, once you have virtual memory enabled, it should be located between `USER_START_VMA` and `USER_END_VMA`. If using `UMODE` and virtual memory, be sure to enable the `UMODE` flag within `usr/Makefile`. You will need to manually tell `gdb` that this is where the program starts.

To do this, open `gdb` as normal with `make debug`, then use the `add-symbol-file` command to add your user program. An example would be

```
add-symbol-file ../user/bin/hello 0x0000000080100000
```

If there is a prompt, hit "Y" to accept. Now, you should be able to set breakpoints and step through the user program. You will need to run this command every time that you re-open `gdb` — if you find yourself using it constantly, consider adding it to your `.gdbinit`.

11.1.3 Useful gdb Commands

The best way to get good at using `gdb` is to actually use it. To get you started, here are some pages with `gdb` commands we found useful. There are a lot more out there, and it is highly encouraged that you spend time looking at the `gdb` man page (`man gdb`) and any other online resources for `gdb`.

Note: Your code may run noticeably slower when using `gdb`, especially if you are performing a lot of comparisons.

- [Stanford CCRMA gdb Guide](#)
- [gdb Wiki](#)

Here are some additional useful gdb tips and tricks:

1. Manipulating Local Variables

You can use arithmetic operations (+, -, *, /, etc.) as well as pointer arithmetic (*, &, [], etc.) on variables in gdb. You can also cast variables or numbers, just like in C. To access the fields of a struct, use `.` or `->` appropriately. This is most useful with `p[rint]`.

2. Watchpoints

You can use watchpoints to "watch" the value of a variable and trigger a breakpoint if it changes. Use `watch <var>` to set a watchpoint on a variable.

3. Conditional Breakpoints and Watchpoints

You can use the format `break WHERE if CONDITION` to set a conditional breakpoint, where it will only happen if a certain condition is met. For example, `watch i if i == 100` would break if `i == 100`. If your variable is constantly being modified, this can cause a major slowdown in your program execution.

4. Printing Registers

Using `i[nfo] r[egisters]` can be useful to see all (or any specific) registers, but all registers can also be referenced with the `$<register name>` variable. For example, `p/x $sp` prints out the value of `sp` in hexadecimal. However, for CSRs, it is recommended you use `i[nfo] r[egisters]` in order to get "pretty printing".

5. gdb History

By adding `set history save on` to your `.gdbinit` file, you can access the history of commands from previous gdb sessions. You can also further customize the file location and history depth.

6. `until` and `finish`

When using gdb, stepping through long for loops can be somewhat annoying. Typically, you will set a breakpoint after the loop and use `c[ontinue]` to skip over it. However, you can also use the `until <line number>` command to skip without having to set a breakpoint.

`fin[ish]` works in a similar way. It runs until the current function exits, then reports the return value of the function. You may find this useful if you accidentally `s[tep]` instead of `n[ext]`.